



Introduction to
Unix systems

Lecture 01

Computer Physics

08 October 2026

Unix-like systems and the command-line environment

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Faculty of Physics and Applied Computer Science,
AGH University of Krakow

Outline

Getting started

Lecture plan and further reading

A brief history of Linux and its role in science

Ways to access and use Linux

Working in the shell: first commands

Documentation and help systems

Filesystem hierarchy

Summary and practice

Section 1

Getting started

Assessment and course credit

100 points available

Assessment	Points
Test 1	35
Test 2	35
Practical tasks	20
UPeL quizzes	10
Total	100

The total is converted into a final grade according to the **AGH Study Regulations**.

Completion requirements

Complete all laboratories and pass the tests. Passing the laboratory classes also grants credit for the lectures.

Regular deadline: the end of classes in the semester.

If you do not pass: a resit covers both tests, worth up to 70 points.

- ▶ Resit scores replace the corresponding original test scores.
- ▶ Practical-task and quiz points are retained.
- ▶ The final grade is recalculated.

Attendance and absences

Lectures: attendance is recommended, but optional.

Laboratories: attendance is mandatory.

Excused absence

- ▶ Independently catch up on the material covered in the missed class.
- ▶ Arrange a date with the instructor to complete an equivalent practical task or take the missed test.

Unexcused absence

- ▶ Any assessed activity scheduled during that class receives **0 points**.
- ▶ The missed activity cannot be completed at a later date.

Resit eligibility: more than **20% unexcused absences** and negative partial academic results may lead to denial of the resit. You may appeal to the course coordinator or the Dean.

Absences and their justification are governed by the applicable AGH regulations.

Your work and the use of AI

Practical work during laboratories

- ▶ Come prepared: review the material indicated by the instructor.
- ▶ Perform the practical tasks and exercises in the Unix/Linux environment using the materials provided.
- ▶ Demonstrate your completed solutions to the instructor.

AI policy

The use of AI tools during laboratory classes is not permitted.

Questions are welcome: ask the instructor whenever you need clarification.

Audiovisual recording of classes requires the instructor's consent.

Section 2

Lecture plan and further reading

Lecture plan

1. **Unix-like systems and the command-line environment:** basic concepts and philosophy of Unix, GNU/Linux systems, the shell and command-line interface, command syntax, documentation and help systems, filesystem hierarchy, paths and navigation.
2. **Files, permissions and the system environment:** file and directory operations, links and metadata, users and groups, ownership and permissions, environment variables, shell configuration, software management and basic system administration.
3. **Data streams, pipelines and text processing:** standard input, output and error streams, redirection and pipelines, regular expressions, searching, filtering and transforming text data using standard Unix tools, including `grep`, `find`, `sed` and `awk`.
4. **Processes and resource management:** the Unix process model, process hierarchy, file descriptors, the concepts of `fork` and `exec`, foreground and background jobs, signals, process monitoring, priorities, system resources and basic memory management.

Lecture plan

5. **Shell scripting and automation:** shell scripts, variables and quoting, command substitution, arguments and exit status, conditional statements, loops and functions, task automation and principles of writing reliable shell scripts.
6. **Remote systems and networked workflows:** remote access using SSH, authentication with cryptographic keys, file transfer and synchronization, basic network tools and diagnostics, persistent terminal sessions and practical work with remote servers and computing clusters.
7. **Version control and reproducible Unix workflows:** basic Git workflow, tracking and reviewing changes, working with repositories and branches, archiving and checksums, reproducibility of computational workflows, and an introduction to containers and isolated environments.

Reading

Required

1. W. Shotts **The Linux Command Line: A Complete Introduction** 3rd ed., No Starch Press, 2026.
2. C. Negus **Linux Bible** 11th ed., Wiley, 2026.
3. B. Ward **How Linux Works: What Every Superuser Should Know** 3rd ed., No Starch Press, 2021.

Optional

1. E. Nemeth et al. **UNIX and Linux System Administration Handbook** 5th ed., Addison-Wesley Professional, 2018.
2. S. Chacon, B. Straub **Pro Git** 2nd ed., Apress, 2014.
3. C. Albing, J. P. Vossen **bash Idioms** O'Reilly Media, 2022.

Learn by playing: the command line

- ▶ **Terminus** — A browser-based text adventure where shell commands let you explore a fictional world. Practise commands such as `ls`, `cd`, `pwd` and `less`. A gentle introduction for complete beginners.
- ▶ **OverTheWire: Bandit** — Connect to a remote server using SSH and solve challenges to unlock the next level. Build confidence with files, permissions, searching and Unix tools. Starts with the basics and gradually becomes harder.
- ▶ **CMD Challenge** — Short command-line challenges with automatic checking in the browser. Practise finding files, processing text and combining commands. Useful for quick exercises after learning the basic shell commands.
- ▶ **The Command Line Murders** — Solve a detective mystery by searching through files and following clues with command-line tools. Download the files and investigate in your own terminal. A good exercise after learning text processing.

Learn by doing: further challenges

- ▶ **RegexOne** — Interactive browser exercises introducing regular expressions, from simple matches to more complex patterns. Useful preparation for text processing with Unix tools; remember that regex syntax varies between tools.
- ▶ **Learn Git Branching** — Solve Git puzzles in a browser with a visual commit graph. Explore commits, branches, merges and rebasing, and see how each command changes the repository history.
- ▶ **VIM Adventures** — A puzzle adventure controlled using Vim commands. Learn keyboard navigation and editing through play. An optional extra for students interested in terminal editors; full access requires a paid licence.
- ▶ **SadServers** — Diagnose and fix problems on real Linux servers through a browser terminal. Best attempted after learning about processes, permissions and system tools. Free scenarios are available; some content requires a paid plan.

Section 3

A brief history of Linux and its role in science

Before Linux: Unix

- ▶ **1969, Bell Labs:** Ken Thompson, Dennis Ritchie and colleagues begin developing Unix.
- ▶ **A shared computing environment:** multiple users run programs, manage files and work interactively.
- ▶ **1973:** Unix is rewritten largely in C, making it easier to adapt to different computers.
- ▶ **A lasting influence:** Unix spreads through universities and research, shaping later operating systems and tools.



Dennis Ritchie and Ken Thompson at a PDP-11.
Photograph: Bell Labs archive.

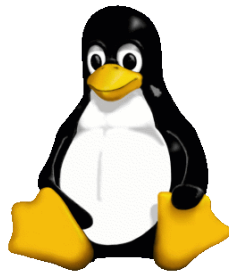
An idea that survived the hardware

Build useful tools that can be combined to solve larger problems.

From Unix ideas to GNU/Linux

- ▶ **1983–1984: GNU** Richard Stallman announces the GNU Project; development begins on a free Unix-like system, including compilers, libraries and utilities.
- ▶ **1991: Linux** Linus Torvalds begins developing a Unix-like kernel. Linux is a new implementation, inspired by Unix.
- ▶ **1992: free software** Linux adopts the GNU General Public License, enabling use, study, modification and redistribution under its terms.
- ▶ **A complete system** The Linux kernel, GNU tools and other software are combined into usable operating systems.

Historical reference: GNU Project: Overview of the GNU System.



Tux, the Linux mascot.
Larry Ewing. Image and credits.

What do we mean by “Linux”?

Scientific applications
Python, numerical solvers, analysis tools

Shell, utilities and libraries
Bash, GNU tools, compilers

Linux kernel
processes, memory, devices

Hardware
CPU, GPU, RAM, storage, network

- ▶ **Kernel:** the core that manages hardware resources and provides services to programs.
- ▶ **Distribution:** a complete system that packages the kernel, tools, libraries and software management.
- ▶ Examples include **Debian, Ubuntu, Fedora and and many, many more.**
- ▶ Many command-line skills transfer between distributions and other Unix-like systems.

Why Linux matters in computational physics

The scientific workload

- ▶ Simulate particles, materials and fields.
- ▶ Analyse experimental data.
- ▶ Repeat calculations for many parameters.
- ▶ Run jobs that need substantial time, memory or parallel computing.

The working environment

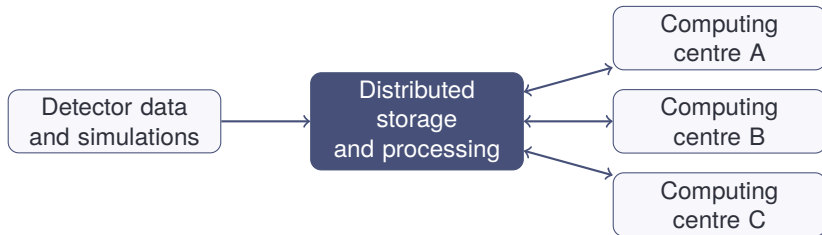
- ▶ Combine compilers, libraries and analysis tools.
- ▶ Automate repeated work with scripts.
- ▶ Access remote machines through SSH.
- ▶ Record commands, software versions and parameters for reproducibility.



Example: automate a temperature sweep in an Ising-model simulation, then collect and plot the results. Linux provides the working environment; reproducibility still requires careful documentation and control of the computational setup.

Example: computing for the Large Hadron Collider

- ▶ Particle physics needs both **experimental data processing** and **simulated events**.
- ▶ The **Worldwide LHC Computing Grid (WLCG)** distributes storage and computing across many institutions.
- ▶ Linux is a central part of CERN's computing environment, including shared services for scientific work.



Conceptual illustration, not the full WLCG architecture.

Connection to this course: files, scripts, remote access and process management become tools for doing physics.

Linux and supercomputers: Ares at Cyfronet

- ▶ A modern supercomputer combines many **compute nodes**, fast networking and shared storage.
- ▶ Programs use parallel algorithms or run many independent calculations.
- ▶ All 500 systems in the **November 2025 TOP500 list** used Linux-family operating systems.
- ▶ **Ares, Cyfronet AGH:** 37 824 CPU cores and 72 NVIDIA V100 GPUs; over 4 PFLOP/s combined theoretical peak performance.

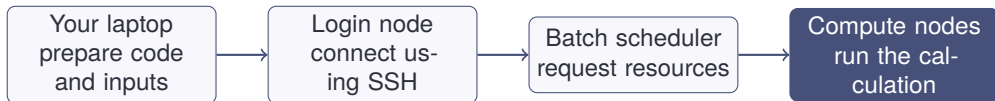


Ares supercomputer.
Photograph: Cyfronet AGH.

1 PFLOP/s = 10^{15} floating-point operations per second. Actual application performance depends on the algorithm, hardware and communication.

Linux statistics: TOP500 operating-system family statistics.

Computing allocations and cluster jobs



- ▶ A **computing grant** provides an allocation of resources for a research project.
- ▶ A job specifies the program and its requirements: execution time, CPU cores, memory and possibly GPUs.
- ▶ The scheduler allocates resources; computationally intensive work runs on compute nodes.
- ▶ Results and logs are inspected, analysed and transferred as needed.

Live demonstration: PLGrid and Ares

A research project, its computing allocation, and an example of how that allocation is used.

Section 4

Ways to access and use Linux

One Linux kernel, many distributions

A **Linux distribution** combines the Linux kernel with system tools, libraries, applications and a way to install and update software.

What can differ?

- ▶ Default desktop and applications.
- ▶ Package manager and software repositories.
- ▶ Release cycle and support period.
- ▶ Configuration tools and default settings.

What carries over?

- ▶ Files, directories and permissions.
- ▶ Shell commands and pipelines.
- ▶ Processes and shell scripts.
- ▶ SSH and remote computing workflows.

For this course: learn the common concepts first. You do not need to try every distribution.

Four widely used distributions

- ▶ **Ubuntu** — A widely used desktop and server distribution with extensive documentation. Its Long-Term Support (LTS) releases are a practical starting point for a development environment. Uses `apt`.
- ▶ **Linux Mint** — Focuses on a comfortable desktop experience. Its main edition is based on Ubuntu and offers a familiar menu-and-panel interface. Uses `apt`.
- ▶ **Debian** — A community-developed distribution. Debian Stable prioritises a consistent, well-tested software environment. It is also the basis of Ubuntu. Uses `apt`.
- ▶ **Fedora Workstation** — A desktop distribution with recent software and a relatively fast release cycle. Suitable for users interested in newer Linux technologies. Uses `dnf`.

The desktop appearance is only one part of a distribution; the underlying command-line skills remain broadly transferable.

Installing Linux or trying it from a USB drive

Linux as the only operating system

Install Linux on the computer's internal drive. It becomes your everyday environment and has direct access to the hardware.

Dual boot: Linux alongside another system

Keep Linux and another operating system on separate partitions or drives. Choose which one to start at boot. Switching systems normally requires a restart.

Live USB / Live CD: try Linux before installing

Boot a live image from removable media. Explore the desktop and test hardware compatibility without installing Linux on the internal drive. Changes are usually lost after shutdown unless persistence has been configured.

Before installing or changing partitions, back up your data. Choose an image compatible with your hardware; installation options differ between PCs and Macs.

Using macOS: a familiar Unix environment

- ▶ macOS has Unix foundations. Open **Terminal** to use commands such as `pwd`, `ls`, `cd`, `cp`, `grep` and `ssh`.
- ▶ Many commands are already available, but their options may differ from the GNU versions commonly used on Linux.
- ▶ Install additional tools using **Homebrew**. Follow its installation instructions, including the shell setup steps.

Example: additional tools via Homebrew

```
brew install bash coreutils gnu-sed
```

GNU tools may have different executable names: for example, `gls` and `gsed`. Installing Bash does not automatically change your shell.

For Linux-specific exercises: connect to Taurus or use a Linux virtual machine.

Homebrew installation GNU coreutils on macOS

Windows: Windows Subsystem for Linux

What is WSL?

Windows Subsystem for Linux lets you run a Linux distribution and its tools alongside Windows applications.

- ▶ **WSL 2 uses a real Linux kernel** inside a lightweight virtual machine managed by Windows.
- ▶ Install a distribution such as Ubuntu or Debian, then use its shell, package manager, compilers and scientific tools.
- ▶ Windows and Linux applications can run at the same time. You do not need to reboot to switch between them.
- ▶ Windows and Linux have separate environments: their paths, installed software and user accounts differ.

A useful option for this course: keep Windows for everyday work and use WSL for local Linux practice.

Microsoft: WSL architecture and versions

Ways to access and use Linux

WSL: installation and first steps

In Windows PowerShell

Open PowerShell **as administrator**:

```
wsl --install -d Ubuntu
```

Restart if requested, then launch Ubuntu and create your Linux username and password.

Check the installed distribution and WSL version:

```
wsl --list --verbose
```

Inside Ubuntu

Update package information and install tools:

```
sudo apt update
```

```
sudo apt install build-essential git
```

Keep Linux projects in your Linux home directory, for example:

```
~/unix-course
```

The Windows C: drive is accessible at `/mnt/c`.

The installation command requires Windows 11 or Windows 10 version 2004/build 19041 or later. WSL 2 requires hardware virtualisation. Store Linux projects in the Linux filesystem for better file-operation performance.

[Installation guide](#) [Working across filesystems](#)

Virtual machines: a complete Linux guest system

A **virtual machine (VM)** runs a guest operating system inside your existing host operating system.

Why use a VM?

- ▶ Explore a complete Linux desktop.
- ▶ Practise installation and administration.
- ▶ Keep a separate experimental environment.
- ▶ Use snapshots to return to an earlier state.

What does it require?

- ▶ RAM, CPU time and disk space shared with the host.
- ▶ Virtualisation software and a Linux image.
- ▶ A suitable guest architecture: typically x86-64 on Intel/AMD, ARM64 on Apple Silicon.
- ▶ Extra setup for some devices and GPU workloads.

Examples: VirtualBox on supported hosts, or UTM on macOS.

Virtualisation runs a compatible CPU architecture. Emulating a different architecture is possible with some tools, but can be much slower.

Our shared Linux environment: Taurus

Taurus at AGH

`taurus.fis.agh.edu.pl`

A Debian Linux server supporting teaching at the Faculty of Physics and Applied Computer Science. We will use it during this course.

- ▶ Connect through **SSH** from your computer. After logging in, commands run on Taurus, and files belong to the remote environment.
- ▶ Local Linux, macOS and Windows with an SSH client can all provide access.
- ▶ The faculty instructions require an initial SSH login to create your home directory for use in the Linux labs.

Connection example

```
ssh your-login@taurus.fis.agh.edu.pl
```

Replace `your-login` with your assigned username.

WFILS AGH: Taurus access and account information

Section 5

Working in the shell: first commands

Terminal, shell and command

- ▶ The **terminal** provides a text interface for input and output.
- ▶ The **shell** interprets the command line. Our examples use Bash on Linux.
- ▶ A command may be a shell builtin, such as `cd`, or an external program, such as `ls`.
- ▶ The **current working directory** is the starting point for relative paths. It belongs to a process; separate shells can work in different directories.

```
student@taurus:~$ pwd  
/home/student
```

The prompt above is illustrative. Type only `pwd`, not the prompt or the displayed output.

Anatomy of a command line

A common pattern is

`command [options] [operands]`

Square brackets in documentation indicate optional parts; do not type them literally.

```
1 ls -l -a /tmp
2 ls -la /tmp
3 head -n 5 /etc/passwd
```

- ▶ `ls`, `head`: command names.
- ▶ `-l`, `-a`: options that change behaviour.
- ▶ `-n 5`: an option with an argument.
- ▶ `/tmp`, `/etc/passwd`: operands naming files or directories. Exact syntax depends on the command.

Many commands accept `--` to end option processing: `ls -- -example.txt` treats `-example.txt` as a filename.

Words, quoting and echo

Unquoted spaces separate words. Quotes group text into one word and control how the shell interprets it.

```
1 echo Hello physics
2 echo "Hello physics"
3 echo "$HOME"
4 echo '$HOME'
```

- ▶ The first two commands display the same text, but pass different numbers of arguments to `echo`.
- ▶ Double quotes allow variable expansion: `$HOME` becomes your home-directory path.
- ▶ Single quotes preserve the literal text `$HOME`.
- ▶ Quote paths containing spaces, for example `cd "simulation results"`.

`echo` normally writes its arguments followed by a newline. For portable scripts and precise formatting, use `printf` rather than relying on `echo -e`.

Finding help and editing the command line

```
1 man ls
2 ls --help
3 help cd
```

- ▶ **man**: reference documentation for a command. In the usual pager, press **q** to leave.
- ▶ **--help**: a quick summary supported by many GNU commands; it is not universal.
- ▶ **help**: documentation for Bash builtins.
- ▶ **Tab**: complete a command or path; pressing it again may show alternatives.
- ▶ **Up/Down**: recall and edit earlier commands.

Read the synopsis and the relevant option

You do not need to memorise every flag. Learn how to check exactly what a command will do.

Paths: locating objects in the filesystem

Paths identify the files and directories on which commands operate.

```
/
|-- etc/
|-- home/
|   |-- student/
|       |-- project/
|           |-- data/
|           |-- results/
|-- tmp/
```

Simplified example hierarchy.

Do not confuse: / is the filesystem root; /root is usually the administrator's home directory.

- ▶ **Absolute path:** starts at /, e.g. /home/student/project/data.
- ▶ **Relative path:** starts at the current directory, e.g. data or ../results.
- ▶ **..:** current directory; **..:** parent directory.
- ▶ In ~ or ~/path, the unquoted tilde expands to your home-directory path.
- ▶ Names are normally case-sensitive on Linux: Data and data differ.

Navigation: pwd and cd

`pwd` prints the current working directory. `cd` changes the current directory of your shell.

```
1 pwd
2 cd /tmp
3 pwd
4 cd ..
5 cd
6 cd -
```

- ▶ `cd /tmp`: go to an absolute path.
- ▶ `cd ..`: go to the parent directory.
- ▶ `cd` without an argument: go home.
- ▶ `cd -`: return to the previous working directory.

Check: from `/home/student/project/data`, where does `cd ../results` take you?

Symbolic links can distinguish logical and physical paths; `pwd -P` reports the physical path.

Inspecting directory contents: ls

```
1 ls
2 ls -l
3 ls -a
4 ls -lh /tmp
5 ls -ld /tmp
```

- ▶ No operand: list the current directory.
- ▶ `-l`: long format, including type, permissions, link count, owner, group, size and modification time.
- ▶ `-a`: include names starting with a dot, including `.` and `...`
- ▶ `-h`: human-readable sizes when used with a size-reporting format such as `-l`.
- ▶ `-d`: show the directory entry itself, rather than its contents.

A leading dot is a naming convention for hidden entries; it does not make a file private or protect it.

Seeing the hierarchy: tree

`tree` displays directory contents recursively.

```
1 tree -L 2 .  
2 tree -d -L 2 .  
3 tree -a -L 2 .
```

- ▶ `-L 2`: limit the display to two levels below the starting directory.
- ▶ `-d`: show directories only.
- ▶ `-a`: include hidden entries.
- ▶ `.`: explicitly start at the current directory.

A useful scientific project layout

Separate input data, source code, results and notes. A directory tree makes that organisation visible.

`tree` is an additional package and may not be installed. On a managed server, follow the local software-installation policy.

Creating directories and files: mkdir and touch

The following examples form a continuous file-management exercise. Begin with a dedicated practice directory in your home directory.

```
1 mkdir -p ~/unix-lab
2 cd ~/unix-lab
3 mkdir data results notes
4 mkdir -p simulations/run01/output
5 touch notes/session.txt
```

- ▶ `mkdir`: create one or more directories.
- ▶ `mkdir -p`: also create missing parent directories; existing directories are accepted.
- ▶ `touch`: update access and modification timestamps. If the file does not exist, normally create an empty file.
- ▶ **`touch` does not erase an existing file.**

Use a fresh practice directory for this walkthrough. Subsequent file examples assume you are in it.

Redirecting standard output

Programs normally have standard input (stdin), standard output (stdout) and standard error (stderr).

```
1 echo "First observation" > notes/session.txt
2 echo "Second observation" >> notes/session.txt
```

- ▶ `>`: send stdout to a file. Create it if absent; with normal Bash settings, truncate it if it exists.
- ▶ `>>`: append stdout to the end; create the file if it does not exist.
- ▶ The **shell** processes these operators; they are not arguments passed to `echo`.
- ▶ Neither operator redirects stderr by default: an error message may still appear in the terminal.

Order matters

The shell opens the output file before the command runs. Never use the same file as input and a > destination: `cat data.txt > data.txt` destroys the original contents.

A small dataset for our examples

Create a text file containing an illustrative measurement series.

```
1 echo "# time_s position_m" > data/motion.txt
2 echo "0.0 0.00" >> data/motion.txt
3 echo "0.1 0.05" >> data/motion.txt
4 echo "0.2 0.20" >> data/motion.txt
5 echo "0.3 0.45" >> data/motion.txt
6 echo "0.4 0.80" >> data/motion.txt
```

- ▶ The first command starts a new file; the others append.
- ▶ The file has six lines: one header and five data rows.
- ▶ The quoted # is written into the file. Whether it marks a comment depends on the program reading it.
- ▶ The shell writes text; it does not interpret these columns as physical quantities.

Predict: what happens if the final command uses > instead of >>?

Printing text: cat and tac

```
1 cat data/motion.txt
2 cat -n data/motion.txt
3 tac data/motion.txt
4 cat notes/session.txt data/motion.txt
```

- ▶ `cat`: concatenate input files in argument order and write them to stdout.
- ▶ With one file, this is a simple way to display its contents.
- ▶ `-n`: add line numbers to the displayed output.
- ▶ `tac`: with its default separator, print lines in reverse order; characters within each line stay unchanged.
- ▶ These commands do not modify their input files.

Prefer a pager for long text files. Avoid printing arbitrary binary files to the terminal. `tac` may need additional installation on macOS.

Reading interactively: less and more

```
1 less data/motion.txt
2 more data/motion.txt
```

A **pager** lets you read text a screen at a time. It does not edit the file.

Key in less	Action
Space / b	Next / previous screen
Enter	Move forward one line
g / G	Beginning / end of file
/pattern, then Enter	Search forward
n / N	Repeat / reverse the search
q	Quit the pager

`more` is a traditional pager: Space advances a screen and `q` quits. Extra features depend on the implementation. Use `less` as your usual interactive text viewer.

Inspecting the beginning and end: head and tail

```
1 head -n 3 data/motion.txt
2 tail -n 2 data/motion.txt
3 tail -n +2 data/motion.txt
4 tail -f notes/session.txt
```

- ▶ `head -n 3`: first three lines, including the header.
- ▶ `tail -n 2`: last two lines.
- ▶ `tail -n +2`: start at line 2 and continue to the end.
- ▶ Without `-n`, both normally show ten lines, or the whole file if it is shorter.
- ▶ `tail -f`: display the end, then keep waiting for appended text. Press **Ctrl+C** to stop following.

Scientific use: inspect a data header, check the final iterations, or monitor an existing simulation log.

Copying files and directories: cp

The source remains in place; the destination receives a copy.

```
1 cp data/motion.txt data/motion-copy.txt
2 cp data/motion.txt results/
3 cp -R data data-backup
4 cp -i data/motion.txt results/motion.txt
```

- ▶ A new destination filename gives the copy a new name.
- ▶ An existing destination directory receives a file with the source basename.
- ▶ `-R`: recursively copy a directory and its contents. Here `data-backup` is assumed not to exist yet.
- ▶ An existing destination file may be overwritten; `-i` asks before overwriting it.

Check the destination: if `data-backup` already exists as a directory, the third command copies `data` inside it instead.

Moving and renaming: mv

Renaming and moving use the same command.

```
1 mv data/motion-copy.txt data/trial.txt
2 mv data/trial.txt results/
3 mv -i results/trial.txt results/check.txt
```

- ▶ Within the same directory: change the name.
- ▶ Into another directory: change the location.
- ▶ After a successful move, the source path is gone.
- ▶ Directories can also be moved; no recursive option is needed.
- ▶ `-i`: ask before overwriting an existing destination.

Within one filesystem, moving usually changes directory entries rather than copying the file's data. Across filesystems, it requires copying the data and removing the source after success.

Removing objects: `rm` and `rmdir`

```
1 mkdir scratch
2 touch scratch/test.txt
3 rm -i scratch/test.txt
4 rmdir scratch
```

- ▶ `rm`: remove file entries. With `-i`, confirm the removal interactively.
- ▶ `rmdir`: remove an empty directory; refuse if it still contains entries, including hidden files.
- ▶ `rm -r directory`: recursively remove a directory and its contents. Verify the path before using it.
- ▶ Ordinary `rm` does not move files to a desktop Trash and provides no general undo operation.

Deletion is not secure erasure

Removing a filename does not guarantee that the underlying data has been physically overwritten.

Controlling the terminal: clear and Ctrl+C

```
1 clear
2 tail -f notes/session.txt
```

- ▶ `clear` clears the terminal display; it does not delete files or erase shell history.
- ▶ The second command continues waiting for new lines. Press **Ctrl+C** to interrupt it.
- ▶ With normal terminal settings, Ctrl+C sends SIGINT to the foreground process group. Programs may handle this signal differently.
- ▶ At a Bash prompt, Ctrl+C cancels the line being entered.

An interrupt is not an undo

Work already performed, including writes to files, is not automatically rolled back. In `less`, use `q` to quit.

Account and session commands: passwd and exit

Change your password

`passwd`

- ▶ Normally changes the current user's password.
- ▶ Follow the current/new password prompts.
- ▶ Typed password characters normally remain invisible.
- ▶ Managed accounts may require a university portal or another local procedure.

Leave the current shell

`exit`

- ▶ Ends the current shell.
- ▶ In a normal SSH session, logs out of the remote host.
- ▶ In a nested shell, returns to the parent shell.
- ▶ Does not shut down the computer.

Always identify the current host: after SSH login, account operations and file commands act on the remote environment.

Interpreting command errors

No such file or directory Check `pwd`, spelling, capitalisation and parent directories.

Permission denied Check the account and permissions. On a shared system, some locations are intentionally inaccessible.

Command not found Check spelling and whether the command is installed and available through the shell's command search path.

Directory not empty Inspect with `ls -a`; hidden files count too.

Verifying successful execution

Commands such as `mkdir`, `cp` and `mv` are normally silent on success. Verify the result with `ls`, `cat` or `pwd`.

Section 6

Documentation and help systems

man: the system reference manual

Reference documentation lets us verify command syntax and behaviour. `man` displays installed reference pages for commands, functions, file formats and broader concepts.

```
1 man ls
2 man man
```

Heading	What to look for
NAME	The topic and a short description.
SYNOPSIS	Command syntax or a function declaration.
DESCRIPTION	Behaviour and important details.
OPTIONS	Available flags and their arguments.
EXIT STATUS	How a command reports success or failure.
FILES	Relevant configuration or data files.
SEE ALSO	Related pages, identified by name and section.

Not every page has every heading. Installed pages can be read offline and normally correspond to the software on that machine.

Manual sections and reference notation

A reference such as **passwd(5)** identifies both a topic and a numbered section of the manual.

Section	Type of documentation	Examples
1	User commands	<code>ls(1)</code> , <code>passwd(1)</code>
2	System calls	<code>open(2)</code> , <code>read(2)</code>
3	Library functions	<code>printf(3)</code> , <code>sqrt(3)</code>
4	Devices and special files	<code>null(4)</code>
5	File formats and conventions	<code>passwd(5)</code> , <code>fstab(5)</code>
6	Games	Installed games' manuals
7	Overviews and conventions	<code>signal(7)</code> , <code>regex(7)</code>
8	System administration commands	<code>useradd(8)</code>

Two meanings of “section”

Section 5 is a category of manual pages. DESCRIPTION is a heading inside an individual page. Neither is a software version number.

Additional sections and suffixes vary by system; not all example pages are installed everywhere.

Command and file-format documentation: passwd

passwd(1): the command

`man 1 passwd`

How to change an account password: command syntax, options and account restrictions.

Reading this page does not run `passwd` or change a password.

An illustrative record in `/etc/passwd`:

```
student:x:1000:1000:Student:/home/student:/bin/bash
```

Seven colon-separated fields: login name, password field, UID, GID, user information, home directory and login shell.

`x` normally indicates that the password hash is stored separately in `/etc/shadow`. To inspect the actual account file, use `less /etc/passwd`.

passwd(5): the file format

`man 5 passwd`

How records in `/etc/passwd` are organised: field meanings and account information.

It documents the format, not your machine's current file contents.

Reading a synopsis and navigating the page

Example synopsis

```
ls [OPTION]... [FILE]...
```

- ▶ `ls`: the literal command name.
- ▶ `OPTION`, `FILE`: placeholders for actual arguments.
- ▶ `[...]`: an optional part.
- ▶ The ellipsis means the preceding item can be repeated.

A concrete invocation:

```
ls -l -a /tmp /etc
```

Reading strategy: identify the topic, check the syntax, find the relevant option, then check limitations and related pages.

When man uses less as its pager

Space / b	Next / previous screen
g / G	Beginning / end
/pattern	Search; press Enter
n / N	Next / previous match
q	Quit the page

Search for an option name or heading, then read the surrounding explanation.

Follow references in SEE ALSO with, for example, `man 5 passwd`.

Finding the right manual page

Without a section number, `man` normally displays the first match in its configured search order. Specify the section when the distinction matters.

```
1 man -f passwd
2 man -k password
3 man -a passwd
4 man 5 passwd
```

- ▶ `man -f passwd`: list short descriptions for that name; also available as `whatis passwd`.
- ▶ `man -k password`: search names and descriptions; also available as `apropos password`.
- ▶ `man -a passwd`: view all matching pages in turn.
- ▶ `man 5 passwd`: open the file-format page directly.

Missing results may mean missing documentation or an incomplete search index. Bash builtins also have `help`, for example `help cd`.

Quick check: which section documents `printf` as a C library function, rather than a command?

Section 7

Filesystem hierarchy

One directory tree, rooted at /

The directories used in our examples form part of a wider hierarchy.

```
/
|-- home/
|-- etc/
|-- usr/
|-- var/
|-- tmp/
|-- dev/
|-- proc/
`-- sys/
```

- ▶ The visible hierarchy begins at `/`, the root directory.
- ▶ Other filesystems are attached at directories called **mount points**.
- ▶ A path may lead to a local disk, network storage or a virtual filesystem provided by the kernel.
- ▶ Standard directory names organise files by purpose, rather than by drive letter.

Selected top-level directories.

One tree does not mean one disk

For example, `/home` may reside on a separate disk or a network filesystem while remaining part of the same tree.

System software, configuration and changing data

Directory	Main purpose
<code>/usr</code>	Installed software: commands in <code>/usr/bin</code> , libraries in <code>/usr/lib</code> , shared data in <code>/usr/share</code> .
<code>/usr/local</code>	Software installed locally by the administrator.
<code>/etc</code>	System-wide configuration, e.g. <code>/etc/passwd</code> .
<code>/var</code>	Changing system data: logs, caches and service state.
<code>/opt</code>	Additional, often self-contained application packages.
<code>/boot</code>	Files used during the boot process.

On many modern distributions, `/bin`, `/sbin` and `/lib` are symbolic links into `/usr`. Inspect the actual layout with:

```
1 ls -ld /bin /sbin /lib
```

Remember: `/usr` is a software hierarchy, not the location of your personal home directory.

Home directories and research data

- ▶ **Your home directory:** often `/home/username`. Keep course files, scripts and small datasets in organised subdirectories. Use `~` or `$HOME` instead of assuming a particular path.
- ▶ **/root:** normally the root user's home directory. It is different from the filesystem root, `/`.
- ▶ **/tmp:** temporary files. They may be removed automatically, including during reboot; do not use it as permanent storage.
- ▶ **/mnt and /media:** conventional locations for temporary mounts and removable media. Exact locations vary by system.

```
1 cd ~
2 pwd
3 mkdir -p unix-course/lecture01
```

On shared research systems: use the storage assigned to your project for large datasets, and check quotas, retention and backup policies.

Special filesystems: interfaces to the running system

Directory	What it exposes or stores
/dev	Device interfaces and special files, such as terminals and /dev/null.
/proc	Kernel and process information, e.g. memory statistics and per-process directories.
/sys	Devices, drivers and other kernel objects, including information and control attributes.
/run	Volatile runtime state maintained by services; normally recreated after boot.

Read-only exploration:

```
1 ls -l /dev/null
2 head -n 5 /proc/meminfo
3 ls /sys/class/net
```

A path does not always identify ordinary disk data

Reading a file under /proc or /sys can obtain information generated by the kernel on demand.

Section 8

Summary and practice

Example: inspecting a measurement dataset

Keep the input data unchanged and save derived files separately. Return to the practice directory and use the dataset created earlier.

```
1 cd ~/unix-lab
2 mkdir -p results/preview
3 head -n 3 data/motion.txt > results/preview/first-lines.txt
4 tail -n +2 data/motion.txt > results/preview/values.txt
5 tail -n 1 data/motion.txt > results/preview/last-value.txt
6 echo "Input: data/motion.txt" > results/preview/README.txt
7 echo "Units: seconds and metres" >> results/preview/README.txt
8 ls -lh results/preview
```

- ▶ The original dataset remains in data/.
- ▶ Each command performs one small, explicit operation.
- ▶ The resulting files and notes can be inspected later.

These are building blocks for scripts that process many simulation runs consistently.

Review: predicting command behaviour

1. A file already contains ten lines. What does `touch file.txt` change?
2. How do `echo done > log.txt` and `echo done >> log.txt` differ?
3. What happens to the source after `cp`? What happens after a successful `mv`?
4. Why might `rmdir results` fail even when `ls results` shows no entries?
5. Which action clears the screen, which interrupts a running command, and which leaves the shell?

Before executing a command, explain its expected effect.

Practice: organise and inspect a measurement file

Work inside `~/unix-lab`. Use a new directory named `practice`.

1. Create `practice/input`, `practice/output` and `practice/scratch`, then change to `practice/`.
2. Create `input/energy.txt` with three lines: 1.0, 1.5 and 2.0.
3. Display the file normally and in reverse line order.
4. Copy it to `output/`, then rename the copy to `energy-copy.txt`.
5. Save its final line to `output/last.txt`.
6. Create an empty file in `scratch/`, remove that file, then remove the empty directory.
7. Verify the resulting structure and confirm that the original data still exists.

Paths in steps 2–7 are relative to `practice/`.

Practice: one possible solution

Starting in ~/unix-lab, with no existing practice directory:

```
1 mkdir -p practice/input practice/output practice/scratch
2 cd practice
3 echo "1.0" > input/energy.txt
4 echo "1.5" >> input/energy.txt
5 echo "2.0" >> input/energy.txt
6 cat input/energy.txt
7 tac input/energy.txt
8 cp input/energy.txt output/
9 mv output/energy.txt output/energy-copy.txt
10 tail -n 1 input/energy.txt > output/last.txt
11 touch scratch/temp.txt
12 rm scratch/temp.txt
13 rmdir scratch
14 ls -l input output
15 cat output/last.txt
```

Expected: the original and its copy each contain three lines; last.txt contains 2.0; scratch/ is gone.